

Learning does not come from watching a demonstration, learning comes from creating the demonstration - Instagram Reel

Intro

Caleb Cohen - <https://www.linkedin.com/in/pittcaleb/>

BS Computer Science from University of Pittsburgh

I have been working as a developer for over 30 years

Have worked for Fortune 100 companies as well as mom & pop software shops with just a handful of devs.

Currently work in AdTech for a public company. We use AWS and Google Cloud for both big data and databases. My team captures about a billion data points a day and turns that into actionable data for use in advertising.

Goal

ETL - Extract, Transform, Load

A lot of behind the scenes work in the corporate jobs I have held involve Python code reading and writing AWS S3 files, modifying AWS database and running entirely inside Docker containers

Have interviewed many people who are excellent coders, but due to their background, simply have zero experience with some of the basics of what we do all day, every day

I want to try to give a relatively quick demo of how easy it is for you to create your own project to perform these tasks and start to give yourself some experience with these technologies.

If I were interviewing you and asked “What is your experience using AWS cloud services” and you simply respond, “I haven’t used them much in college or in my job so far, but independently, I have written code to read files on S3, to use cloud databases and run jobs inside a docker container” you would be leagues ahead of your peers who don’t have that experience. Not to mention I also see your drive to learn things that haven’t been part of your day-to-day life thus far.

In this demo, we’re going to create an AWS Account, Create an S3 Bucket, Create a MySQL Database, Create Access Keys and use these all to read a file in S3 and push the data to this database. If time permits, we will containizerise this code with a docker file, run it locally, and then push it to ECR and run the code in an EC2 instance.

It may sound daunting, but it's not, and that's the entire point of this demo, not to show you a slick well rehearsed demo, but to simply show you these technologies aren't difficult to grasp and to give you a jumping off point to do more on your own. To simply take the fear out of doing this yourself.

Demo Problems

When you watch a demo, you get a sanitized version of how to do something. The true learning comes from creating the demo, something you won't get to experience watching this. Use this as a jumping off point to learn more independently.

The Application

All files referenced in this document can be found on GitHub:

<https://github.com/PittCaleb/CodeCoffeeAWSPythonDemo>

AWS Account

<https://aws.amazon.com/>

I am not going to go through the exercise of how to create an AWS Account
You will need a credit card, but you can keep your costs to zero or close to zero pretty easily. You can link it to your Amazon shopping account, but it will add another level of security when you do so

S3 Bucket

In the console, click or search for S3
Create Bucket, give it a name, ensure it is not public

AWS Security/Permissions

All resources inside AWS are controlled one of two ways: Access Keys or Roles

For this exercise, we're going to use Access Keys since it's much easier to demonstrate and I am not a DevOps expert.

Note: Once keys for your project are "out there" they are out there forever. Never check keys into code on github and never give them more permissions than they need.

IAM

In the AWS Console, click or search for IAM

Policies

Policies, search for s3FullAccess, Click +, copy the JSON

Create policy, click JSON, paste in above. Where it shows "*" for Resource, change to the bucket we created above, i.e.:

```
"Resource": [  
    "arn:aws:s3:::codecoffeawsprimer",  
    "arn:aws:s3:::codecoffeawsprimer/*"  
]
```

Give the policy a name, CodeCoffeePrimerS3FullControl

Users

Users, Create User, give the user a name, and select Attach Policies Directly. Type in the name of the policy created above in the search window, select it, next, Create user

Select the User and in the Permissions tab, type in the name of the permission we just created

Select the User again, Security Credentials, Create Access Keys, Ignore the warnings
Your corporate DevOps will direct you differently if that's what they want you to do in a job

Copy the keys to your code for future use

Synopsis

What we have done is created an AWS Security policy granting people who have it full access to a specific S3 bucket, then we created a user who was granted this policy permission. Finally, we created access keys for this user so we can authenticate into AWS as this user via code.

Test File

Create a simple test file, shown below on your computer and upload it to S3

AWS Console, S3, Click your bucket, Upload, Drag the file named code_coffee.csv

```
John Smith,123 Main St,Norcross,GA,30092,770-555-1212  
Alice Johnson,456 Oak Ave,Atlanta,GA,30303,404-555-3434  
Robert Williams,789 Pine Ln,Decatur,GA,30030,678-555-5656  
Emily Brown,101 Elm Rd,Marietta,GA,30060,770-555-7878  
David Jones,222 Maple Dr,Roswell,GA,30076,470-555-9090  
Sarah Garcia,333 Willow Ct,Smyrna,GA,30080,678-555-2121  
Michael Rodriguez,444 Birch St,Duluth,GA,30096,770-555-4343
```

```
Jessica Martinez,555 Cedar Ave,Lawrenceville,GA,30043,470-555-6565
Christopher Wilson,666 Oak Tree Ln,Peachtree
Corners,GA,30092,770-555-8787
Ashley Taylor,777 Pinecrest Rd,Suwanee,GA,30024,678-555-0909
```

Python Code, Part I

code_coffee_s3_read.py

We're going to be using boto3, which is the official AWS SDK for files

If you need to, you will need to pip install it to your system:

Pip install boto3

```
import boto3
import csv
import io

AWS_ACCESS_KEY = 'xxx'
AWS_SECRET_KEY = 'yyy'
AWS_BUCKET_NAME = 'codecoffeawsprimer'

def get_file():
    s3_client = boto3.client('s3', aws_access_key_id=AWS_ACCESS_KEY,
aws_secret_access_key=AWS_SECRET_KEY)
    response = s3_client.get_object(Bucket=AWS_BUCKET_NAME,
Key='code_coffee.csv')
    csv_content = response['Body'].read().decode('utf-8')
    csv_file = io.StringIO(csv_content)
    csv_reader = csv.reader(csv_file)

    for row in csv_reader:
        print(row)

if __name__ == '__main__':
    get_file()
```

This code will read the file from CSV then convert it into an IO object so we can use the CSV Reader module to auto-parse each line for us.

Part 1 of the exercise is complete. We created a file in S3 and were able to write code to read the file.

Boto3 S3 Documentation:

<https://boto3.amazonaws.com/v1/documentation/api/latest/reference/services/s3.html>

Now we need to write this data to a database. Let's jump back to the AWS Console

Database Administration

Create Database

Back in the AWS Web Console, search for MySQL and click on the Aurora/RDS Link
Aurora is Amazon's proprietary but compatible MySQL database engine

Create a database: You'll notice a bunch of options. We will chose MySQL and then Free Tier

Give it a name, such as codecoffeedb

Note the default admin username is admin, provide a password (codecoffeedbpwd)

Notice the size is db.t4g.micro, in real environments, this will be much larger. You can look around and see all the different configuration options. As long as you don't click create, you poke around and learn.

For this, we'll select Public Access: Yes

In a production environment, it is normal to not allow public access, but only allow from inside a specific VPC, corporate VPN or only directly from a server for security purposes.

For this test, we don't want any logging to keep our costs down/zero
Click create database

Takes about 7 mins to come online, short break, Q/A's

We now need to tell AWS that we want to access this DB from anywhere on the Internet. You would never do this in production, but for this exercise, we will do so:

VPC Security Groups, Inbound rules, edit inbound rules, MySQL / Anywhere IPv4, Save Rules

Set Up the Database

In AWS, select the database and grab the endpoint

In your DB browser, connect to the endpoint using the username/password we assigned earlier

In your SQL console, create the database and a table:

```
create database codecoffee;

create table users
(
  id          int AUTO_INCREMENT PRIMARY KEY,
  name        varchar(128) null,
  address     varchar(128) null,
  city        varchar(64)  null,
  state       varchar(3)   null,
  zip         varchar(10)  null,
  phone       varchar(16)  null,
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Python Code, Part II

code_coffee_sql_insert.py

We're going to be using the mysql.connector library:

pip install mysql-connector-python

MySQL cursor class documentation:

<https://dev.mysql.com/doc/connector-python/en/connector-python-api-mysqldcursor.html>

Adding to the original code, we add in methods to connect to, write to and read from the database:

```
import boto3
import csv
import io
import mysql.connector

AWS_ACCESS_KEY = 'XXXXX'
AWS_SECRET_KEY = 'YYYYY'
AWS_BUCKET_NAME = 'codecoffeawsprimer'

db_host = "codecoffeedb.cecft67dh8f0.us-east-1.rds.amazonaws.com"
db_user = "XX"
db_password = "YY"
db_name = "codecoffee"
```

```

def get_file():
    s3_client = boto3.client('s3', aws_access_key_id=AWS_ACCESS_KEY,
aws_secret_access_key=AWS_SECRET_KEY)
    response = s3_client.get_object(Bucket=AWS_BUCKET_NAME,
Key='code_coffee.csv')
    csv_content = response['Body'].read().decode('utf-8')
    csv_file = io.StringIO(csv_content)
    csv_reader = csv.reader(csv_file)

    return csv_reader

def open_db():
    mydb = mysql.connector.connect(
        host=db_host,
        user=db_user,
        password=db_password,
        database=db_name
    )

    if mydb.is_connected():
        print(f"Successfully connected to MySQL database: {db_name} on
{db_host}")
        return mydb

    return False

def write_db(csv_file):
    db = open_db()
    mycursor = db.cursor()

    for row in csv_file:
insert_query = f"INSERT INTO users (name, address, city, state, zip,
phone) VALUES ('{row[0]}',
'{row[1]}', '{row[2]}', '{row[3]}', '{row[4]}', '{row[5]}')"

        mycursor.execute(insert_query)
        db.commit()

    mycursor.close()
    db.close()

def read_db():
    db = open_db()
    mycursor = db.cursor()

```

```

query = "SELECT * FROM users"
mycursor.execute(query)
results = mycursor.fetchall()
mycursor.close()
db.close()

return results

if __name__ == '__main__':
    csv_file = get_file()
    write_db(csv_file)

    data = read_db()

    for row in data:
        print(row)

```

Dockerizing the Code

Docker is a way to containerize your code. In general, I would argue all production code at the corporate level is run inside docker containers in 2025.

It allows me to package my code up, with all ancillary requirements, files, keys, etc. - and run it anywhere. I can run it on my laptop, I can send you the docker container and you can run it on your system, or I can push it to a cloud service and run it in AWS or Azure or Google Cloud - without regard to the hardware I am running it on.

Local Docker

Install Docker, chose free local plan:
<https://www.docker.com/pricing/>

Dockerfile

Requirements.txt

```

boto3==1.37.34
mysql-connector-python==9.3.0

```

Dockerfile

```

FROM python:3.9-slim-buster

WORKDIR /app

COPY requirements.txt .

```

```
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# RUN chmod +x code_coffee.py

CMD ["python", "code_coffee.py"]
```

Build

```
docker build -t codecoffee .
```

Build to target Linux: *(necessary to run on EC2 is built on mac)*

```
docker buildx create --name mydriver --driver docker-container --use
docker buildx build --platform linux/amd64 -t [acct
id].dkr.ecr.[region].amazonaws.com/codecoffee/demo-test:latest --push .
```

Test locally

Go to Docker dashboard and run the code

Deployed Docker

We need to put this docker container somewhere that can be used “in the cloud.” ECR = Elastic Container Registry

Push to ECR

In the AWS console. Go to ECR and create a new private repo.

In the terminal, using the [AWS CLI](#) , authenticate docker to your AWS instance:

```
aws ecr get-login-password --region us-east-1 | docker login --username
AWS --password-stdin [account_id].dkr.ecr.[region].amazonaws.com
```

Give your docker image the latest tag:

```
docker tag codecoffee
[account_id].dkr.ecr.[region].amazonaws.com/codecoffee/demo-test:latest
```

Push your image to the ECR:

```
docker push
[account_id].dkr.ecr.[region].amazonaws.com/codecoffee/demo-test:latest
```

```
(.venv) → codecoffee docker tag codecoffee 064446944801.dkr.ecr.us-east-1.amazonaws.com/codecoffee/demo-test:latest
(.venv) → codecoffee docker push 064446944801.dkr.ecr.us-east-1.amazonaws.com/codecoffee/demo-test:latest
The push refers to repository [064446944801.dkr.ecr.us-east-1.amazonaws.com/codecoffee/demo-test]
426b6a3baf38: Pushed
4e25f127b811: Pushing [=====>] 44.34MB/127.5MB
4c4b6f3b69d4: Pushed
2b554972eaab: Pushed
e563f3050990: Pushed
cd49bcee103a: Pushed
6e4f4f2a1b40: Pushed
dcbd2252343f: Pushed
87bc36f72cf5: Pushing [=====>] 41.06MB/63.48MB
```

In AWS, you can confirm the image has been successfully pushed to the cloud.

EC2 Instance

Back in the AWS web UI, go to EC2, Elastic Container

Launch an instance:

- Give it a name (codecoffeedemo)

- Choose a free tier eligible server

- Give keypair name, save file for SSH access

Connect to the EC2 Instance using the Web interface

Install docker on the EC2 Instance:

```
sudo yum install docker -y
```

Start docker:

```
sudo systemctl start docker
```

AWS Web UI

IAM / Users, Permissions, Create access key pair, CLI

AC2 CLI Again

Use those creds in the EC2 console to authenticate with AWS CLI and then

Docker:

```
aws configure
```

```
aws ecr get-login-password --region us-east-1 | docker login
--username AWS --password-stdin [acct
id].dkr.ecr.[region].amazonaws.com
```

Configure user account to have access to Docker:

```
Whoami (ec2-user)
```

```
groups ec2-user (notice no docker)
```

```
sudo usermod -aG docker ec2-user
```

```
newgrp docker
```

Pull Docker Image:

```
docker pull [acct  
id].dkr.ecr.[region].amazonaws.com/codecoffee/demo-test:latest
```

Finally, run the image:

```
docker run [acct  
id].dkr.ecr.[region].amazonaws.com/codecoffee/demo-test:latest
```

See the output on the screen that it ran and check the database that the new data has been inserted

Next Steps

Take this silly demo and turn it into an application that does something real.

Read a file you scraped from the CDC Website and push it into your database, then write some code to analyze it for example.

This should just be the start of your AWS journey, it is just meant to show you how to connect some Python code to AWS S3 and an AWS RDS instance and run code on a deployed server. The next steps are your own to take.

I am more than happy to work with you as you write your own code, to answer questions and to guide you as you need help.